Synthetic Reasoning

Abstract Symbols / Natural Language

合成した規則と事実で、推論の基本操作を評価する

sasakiy84

1. 記号列と自然言語の実例
2. 生成方法と問題の構成
3. 答えの形式と問題の特徴
4. 先行研究と公開生成器

記号列と自然言語の実例

規則を見つけ、適用し、元の規則を推定する

- Stanford CRFM 研究チームが 2022 年に設計した、推論の基本操作を調べる合成課題。抽象版は記号列の照合・変数置換・規則の帰納を扱う。
- 自然言語版は、条件文の規則と一つの事実から新しい属性を導く。 easy と hard があり、 hard は上位カテゴリや語の言い換えも使う。
- 二つの版は別の生成器で作る。それぞれの形式で、規則を読み取って適用できるかを調べる。

[1] Liang et al. (2023). Holistic Evaluation of Language Models. TMLR.
https://arxiv.org/abs/2211.09110

[2] Stanford CRFM. Synthetic Reasoning — 公開生成器。
https://github.com/stanford-crfm/helm/tree/v0.4.0

実例 1 ：変数置換

問題（全文）

> Rules: * Y - * X Z | Substitutions: X by " shark " , Y by " stork " , Z by " stork "

正解： * stork - * shark stork

日本語訳（記号・置換文字列は原文のまま）

規則： * Y - * X Z
置換： X を " shark " 、 Y を " stork " 、 Z を " stork "

[1] Stanford CRFM. SyntheticReasoningScenario — 記号列の合成課題。
https://github.com/stanford-crfm/helm/blob/v0.4.0/src/helm/benchmark/scenarios/synthetic_reasoning_scenario.py

記号は計算せず、指定どおりに置き換える

- 実例 1 では X が shark 、 Y と Z が stork に対応する。規則列にある変数をすべて置換し、順番と記号を保って出力する。
- + 、 - 、 * 、 = は、文字列を組み立てるための記号として使う。
- 一つの変数に 1 語だけでなく 2 語の文字列を割り当てる例もある。変数の反復位置と、置換する文字列の範囲を対応させる。

[1] Stanford CRFM. SyntheticReasoningScenario — 記号列の合成課題。
https://github.com/stanford-crfm/helm/blob/v0.4.0/src/helm/benchmark/scenarios/synthetic_reasoning_scenario.py

実例２：規則の照合

問題（全文）

Rules: = Z + - X | - + = X Z | Z - X + = | + - = Z X | Result: = watermelon lion + - zebra

正解： = Z + - X

日本語訳（記号・置換文字列は原文のまま）

候補となる規則： = Z + - X | - + = X Z | Z - X + = | + - = Z X
置換後の文字列： = watermelon lion + - zebra

[1] Stanford CRFM. SyntheticReasoningScenario — 記号列の合成課題。
https://github.com/stanford-crfm/helm/blob/v0.4.0/src/helm/benchmark/scenarios/synthetic_reasoning_scenario.py

4 候補から、置換後の列に合う規則を選ぶ

- 実例 2 は、 4 つの規則候補と、置換後の文字列を入力にする。正解として番号ではなく、合致する規則列そのものを出力する。
- 帰納課題では、候補や置換辞書を示さず、同じ規則から生成した 2 つの結果だけを見せる。出力は Rule: に続く元の規則列である。
- この 3 課題は同じ形式の記号列を使うが、与えられる情報が異なる。置換では辞書が既知、照合では規則候補が既知、帰納では規則を推定する。

[1] Stanford CRFM. SyntheticReasoningScenario — 記号列の合成課題。
https://github.com/stanford-crfm/helm/blob/v0.4.0/src/helm/benchmark/scenarios/synthetic_reasoning_scenario.py

実例３：自然言語・ hard

問題（全文）と日本語訳

If Dan is young, then Dan is sad.
If Dan is scary, then Dan is hot.
If person is purple or small, then person is beautiful.
If Dan is clean, then Dan is green.
If Dan is bad, then Dan is soft.
Fact:
Dan is tiny and cool.
The following can be determined about Dan:

正解： Dan is beautiful.

Dan が若ければ、 Dan は悲しい。
Dan が恐ろしければ、 Dan は熱い。
人が紫色であるか、または小さいなら、その人は美しい。
Dan が清潔なら、 Dan は緑色。
Dan が悪ければ、 Dan は柔らかい。
事実：
Dan はとても小さく、涼しい。
Dan について何が導けるか。

Dan は美しい。

[1] Stanford CRFM. SRNScenario — 自然言語の合成課題。
https://github.com/stanford-crfm/helm/blob/v0.4.0/src/helm/benchmark/scenarios/synthetic_reasoning_natural_scenario.py

hard では、言い換えと上位カテゴリを対応させる

- 実例 3 では、事実の tiny を規則の small へ、 Dan を人の規則へ対応させる。 purple または small という条件の small が満たされるため、 beautiful を導ける。
- cool は生成器の語彙表では cold の具体的な表現だが、この例の 5 規則から cold に基づく新しい属性は導けない。答えには beautiful だけを含める。
- 規則の帰結は、ほかの規則の条件へ再利用されないように作られる。自然言語版は、規則を何段も連鎖させる課題ではなく、1 段の適用を評価する。

[1] Stanford CRFM. SRNScenario — 自然言語の合成課題。
https://github.com/stanford-crfm/helm/blob/v0.4.0/src/helm/benchmark/scenarios/synthetic_reasoning_natural_scenario.py

生成方法と問題の構成

抽象版は短い列と置換辞書から生成する

要素	生成条件
変数	X / Y / Z から、重複を許して 2 〜 3 個
記号	+ / - / * / = から、重複を許して 2 〜 3 個
置換語彙	動物 12 語と果物 10 語の計 22 語
置換文字列	各変数に 1 〜 2 語を割り当てる

変数と記号をシャッフルして 4 〜 6 要素の規則列を作る。同じ変数は同じ文字列に置換される。

[1] Stanford CRFM. SyntheticReasoningScenario — 記号列の合成課題。
https://github.com/stanford-crfm/helm/blob/v0.4.0/src/helm/benchmark/scenarios/synthetic_reasoning_scenario.py

自然言語版は 5 規則と 2 属性の事実を作る

- 人・動物・植物の 3 カテゴリから対象を選ぶ。 25 の一般属性を使い、 1 属性の条件か、 and/or で結んだ 2 属性の条件から、別の 1 属性を導く規則を 5 つ作る。
- 事実には 2 つの属性を与える。満たされる規則の帰結を集め、対象がそれらを持つという文を正解にする。新たな帰結がなければ Nothing. となる。
- easy は規則と事実で同じ対象・属性の語を使う。 hard では対象を person などのカテゴリで表す規則や、 tiny / small などの語彙対応を使う。

[1] Stanford CRFM. SRNScenario — 自然言語の合成課題。
https://github.com/stanford-crfm/helm/blob/v0.4.0/src/helm/benchmark/scenarios/synthetic_reasoning_natural_scenario.py

既定では、設定ごとに 11,000 問を作る

分割	件数
train	1,000
valid	5,000
test	5,000

置換・照合・帰納と、自然言語 easy/hard をそれぞれ生成する。問題数と乱数の種は指定できる。

[1] Stanford CRFM. Synthetic Reasoning — 公開生成器。
https://github.com/stanford-crfm/helm/tree/v0.4.0

機械的な正解を、生成条件と一緒に保存できる

- 抽象版は、規則列への置換処理から正解を作る。自然言語版は、事実が各規則の条件を満たすか調べ、その帰結をまとめて正解文を作る。
- 短い語彙表と文型を使い、規則や事実を機械的に組み合わせて多数の問題を作れる。
- 同じ乱数 seed で生成器を初期化すると、同じ入力と正解を再生成できる。

[1] Stanford CRFM. SyntheticReasoningScenario — 記号列の合成課題。
https://github.com/stanford-crfm/helm/blob/v0.4.0/src/helm/benchmark/scenarios/synthetic_reasoning_scenario.py
[2] Stanford CRFM. SRNScenario — 自然言語の合成課題。
https://github.com/stanford-crfm/helm/blob/v0.4.0/src/helm/benchmark/scenarios/synthetic_reasoning_natural_scenario.py

答えの形式と問題の特徴

課題ごとに、求める出力の形が変わる

- 置換では、記号を保った置換後の文字列を答える。照合では候補の規則列そのもの、帰納では Rule: に続く規則列を出力する。
- 自然言語版は、事実から新たに導ける属性をまとめる。複数なら and で結び、新たな帰結がなければ Nothing. と答える。

[1] Stanford CRFM. SyntheticReasoningScenario — 記号列の合成課題。
https://github.com/stanford-crfm/helm/blob/v0.4.0/src/helm/benchmark/scenarios/synthetic_reasoning_scenario.py
[2] Stanford CRFM. SRNScenario — 自然言語の合成課題。
https://github.com/stanford-crfm/helm/blob/v0.4.0/src/helm/benchmark/scenarios/synthetic_reasoning_natural_scenario.py

抽象版：記号を含む文字列の一致率

モデル	置換 %	照合 %	帰納 %
text-davinci-003 (2022)	95.02	32.30	11.07
OPT-175B (2022)	1.94	25.37	0.00
Llama 2 70B (2023)	83.50	35.53	7.57

2023 年公開の成績。各課題の test 515 問で、記号を含む文字列の一致を採点する。 text-davinci-003 ・ OPT は 3 試行、 Llama 2 は 1 試行。 3 モデルとも、置換より帰納で低い。

[1] Stanford CRFM (2023). 公開成績（ 2023 年 11 月公開版）。 https://storage.googleapis.com/crfm-helm-public/classic/benchmark_output/runs/v0.3.0/synthetic_reasoning%3Amode%3Dinduction%2Cmodel%3Dmeta_llama-2-70b/stats.json

自然言語版：導いた属性集合の F1

モデル	easy %	hard %
text-davinci-003 (2022)	82.58	64.13
OPT-175B (2022)	36.05	13.53
Llama 2 70B (2023)	49.97	44.40

2023 年公開の成績。各難度の test 515 問で、属性の過不足を測る F1 を、各問で求め平均する。 text-davinci-003 ・ OPT は 3 試行、 Llama 2 は 1 試行。 hard では全モデルで低下する。

[1] Stanford CRFM (2023). 公開成績（ 2023 年 11 月公開版）。 https://storage.googleapis.com/crfm-helm-public/classic/benchmark_output/runs/v0.3.0/synthetic_reasoning_natural%3Adifficulty%3Dhard%2Cmodel%3Dmeta_llama-2-70b/stats.json

候補の重複や、変数名も問題の特徴になる

- 帰納問題の参照正解は、生成器が付けた変数名を使う。同じ置換関係でも、変数名を入れ替えると表記が変わる。
- 照合候補には、同じ規則列が重なることや、正解だけが長くなることがある。こうした生成方法から、推論以外の手掛かりが生まれる。
- 自然言語版は、定めた語彙対応と 1 段の条件適用を扱う。

[1] Stanford CRFM. SyntheticReasoningScenario — 記号列の合成課題。
https://github.com/stanford-crfm/helm/blob/v0.4.0/src/helm/benchmark/scenarios/synthetic_reasoning_scenario.py
[2] Stanford CRFM. SRNScenario — 自然言語の合成課題。
https://github.com/stanford-crfm/helm/blob/v0.4.0/src/helm/benchmark/scenarios/synthetic_reasoning_natural_scenario.py

先行研究と公開生成器

記号操作と、自然言語の規則推論を合成する

- LIME （ 2021 ）は、合成した演繹・帰納・仮説推論の課題で事前学習し、数学的推論に役立つ帰納バイアスを学ばせる研究。基本操作を合成する発想が、抽象版の出発点となった。
- Clark ら（ 2020 ）は、自然言語で書いた規則と事実から結論を導く研究を行った。自然言語版は、その考え方を短い語彙と文型で実装した。

[1] Wu et al. (2021). LIME: Learning Inductive Bias for Primitives of Mathematical Reasoning. ICML. https://proceedings.mlr.press/v139/wu21c.html
[2] Clark, Tafjord & Richardson (2020). Transformers as Soft Reasoners over Language. IJCAI. https://arxiv.org/abs/2002.05867

公開生成器で、課題や難度を選ぶ

- 抽象版は SyntheticReasoningScenario で、置換・照合・帰納のモードを選ぶ。自然言語版は SRNScenario で、 easy または hard を選べる。
- どちらも短い語彙表とテンプレートから、問題文と正解を生成する。コードは Apache-2.0 で公開されている。
- 乱数 seed を変えると、同じ形式で異なる規則や事実を持つ問題を作れる。

[1] Stanford CRFM. SyntheticReasoningScenario — 記号列の合成課題。
https://github.com/stanford-crfm/helm/blob/v0.4.0/src/helm/benchmark/scenarios/synthetic_reasoning_scenario.py
[2] Stanford CRFM. SRNScenario — 自然言語の合成課題。
https://github.com/stanford-crfm/helm/blob/v0.4.0/src/helm/benchmark/scenarios/synthetic_reasoning_natural_scenario.py

短い規則と事実で、推論の基本操作を試す

- Synthetic Reasoning は、短い記号列の照合・置換・帰納と、自然言語の 1 段の規則適用を扱う。
- 抽象版は規則や置換後の文字列、自然言語版は新たに導ける属性を答える。与えられる情報が変わると、必要な操作も変わる。
- 語彙と文型が定められているため、形式や難度をそろえ、多数の問題で基本操作を調べられる。

[1] Liang et al. (2023). Holistic Evaluation of Language Models. TMLR. https://arxiv.org/abs/2211.09110

Appendix

1. 追加の生成例
2. 語彙と生成の補足
3. 関連研究と参考文献

実例 4 ：変数置換

問題（全文）

> Rules: X Z X = - - | Substitutions: X by " grape apple " , Z by " penguin "

正解： grape apple penguin grape apple = - -

日本語訳（記号・置換文字列は原文のまま）

規則： X Z X = - -
置換： X を " grape apple " 、 Z を " penguin "

[1] Stanford CRFM. SyntheticReasoningScenario — 記号列の合成課題。
https://github.com/stanford-crfm/helm/blob/v0.4.0/src/helm/benchmark/scenarios/synthetic_reasoning_scenario.py

実例 5 ：変数置換

問題（全文）

> Rules: X = Y X * | Substitutions: X by " peach " , Y by " cobra eagle "

正解： peach = cobra eagle peach *

日本語訳（記号・置換文字列は原文のまま）

規則： X = Y X *
置換： X を " peach " 、 Y を " cobra eagle "

[1] Stanford CRFM. SyntheticReasoningScenario — 記号列の合成課題。
https://github.com/stanford-crfm/helm/blob/v0.4.0/src/helm/benchmark/scenarios/synthetic_reasoning_scenario.py

実例 6 ：規則の照合

問題（全文）

Rules: + = X Y | Y X = + | + Y X = Y | Y X = + | Result: + horse cobra horse = horse

正解： + Y X = Y

日本語訳（記号・置換文字列は原文のまま）

候補となる規則： + = X Y | Y X = + | + Y X = Y | Y X = +
置換後の文字列： + horse cobra horse = horse

[1] Stanford CRFM. SyntheticReasoningScenario — 記号列の合成課題。
https://github.com/stanford-crfm/helm/blob/v0.4.0/src/helm/benchmark/scenarios/synthetic_reasoning_scenario.py

実例 7 ：規則の照合

問題（全文）

> Rules: = Z Y = | Z = = Y | = Z Y = | = Y = Z Y | Result: = seal pear = lion seal pear

正解： = Y = Z Y

日本語訳（記号・置換文字列は原文のまま）

候補となる規則： = Z Y = | Z = = Y | = Z Y = | = Y = Z Y
置換後の文字列： = seal pear = lion seal pear

[1] Stanford CRFM. SyntheticReasoningScenario — 記号列の合成課題。
https://github.com/stanford-crfm/helm/blob/v0.4.0/src/helm/benchmark/scenarios/synthetic_reasoning_scenario.py

実例 8 ：規則の帰納

問題（全文）

> Two results: * stork - * shark stork | * horse - * buffalo penguin horse

正解： Rule: * Y - * X Z

日本語訳（記号・置換文字列は原文のまま）

同じ規則から得た 2 つの文字列： * stork - * shark stork | * horse - * buffalo penguin horse

[1] Stanford CRFM. SyntheticReasoningScenario — 記号列の合成課題。
https://github.com/stanford-crfm/helm/blob/v0.4.0/src/helm/benchmark/scenarios/synthetic_reasoning_scenario.py

実例 9 ：規則の帰納

問題（全文）

> Two results: grape apple penguin grape apple = - - | pear shark eagle pear shark = - -

正解： Rule: X Z X = - -

日本語訳（記号・置換文字列は原文のまま）

同じ規則から得た 2 つの文字列： grape apple penguin grape apple = - - | pear shark eagle pear shark = - -

[1] Stanford CRFM. SyntheticReasoningScenario — 記号列の合成課題。
https://github.com/stanford-crfm/helm/blob/v0.4.0/src/helm/benchmark/scenarios/synthetic_reasoning_scenario.py

実例 10 ：自然言語・ easy

問題（全文）

If a dandelion is small, then the dandelion is good.
If a dandelion is strong, then the dandelion is hot.
If a dandelion is big and soft, then the dandelion is boring.
If a dandelion is red and scary, then the dandelion is old.
If a dandelion is clean or young, then the dandelion is purple.
Fact:
The dandelion is purple and boring.
The following can be determined about the dandelion:

正解： Nothing.

[1] Stanford CRFM. SRNScenario — 自然言語の合成課題。
https://github.com/stanford-crfm/helm/blob/v0.4.0/src/helm/benchmark/scenarios/synthetic_reasoning_natural_scenario.py

実例 10 ：自然言語・ easy （訳）

問題（全文）の日本語訳

タンポポが小さければ、そのタンポポはよい。
タンポポが強ければ、そのタンポポは熱い。
タンポポが大きく、かつ柔らかいなら、そのタンポポは退屈なものだ。
タンポポが赤く、かつ恐ろしいなら、そのタンポポは古い。
タンポポが清潔であるか、または若いなら、そのタンポポは紫色。
事実：
そのタンポポは紫色で、退屈なものだ。
そのタンポポについて何が導けるか。

正解：新たに導ける属性はない。

[1] Stanford CRFM. SRNScenario — 自然言語の合成課題。
https://github.com/stanford-crfm/helm/blob/v0.4.0/src/helm/benchmark/scenarios/synthetic_reasoning_natural_scenario.py

実例 11 ：自然言語・ easy

問題（全文）と日本語訳

If a rose is hot, then the rose is blue.
If a rose is scary or small, then the rose is happy.
If a rose is smart, then the rose is red.
If a rose is slow or green, then the rose is old.
If a rose is bad, then the rose is cold.
Fact:
The rose is beautiful and hot.
The following can be determined about the rose:

バラが熱ければ、そのバラは青い。
バラが恐ろしいか、または小さいなら、そのバラは幸せ。
バラが賢ければ、そのバラは赤い。
バラが遅いか、または緑色なら、そのバラは古い。
バラが悪ければ、そのバラは冷たい。
事実：
そのバラは美しく、熱い。
そのバラについて何が導けるか。

正解： The rose is blue.

バラは青い。

[1] Stanford CRFM. SRNScenario — 自然言語の合成課題。
https://github.com/stanford-crfm/helm/blob/v0.4.0/src/helm/benchmark/scenarios/synthetic_reasoning_natural_scenario.py

実例 12 ：自然言語・ hard

問題（全文）と日本語訳

If a poppy is green and hot, then the poppy is cold.
If a poppy is blue, then the poppy is sad.
If a plant is red, then the plant is bad.
If a poppy is strong, then the poppy is old.
If a plant is small, then the plant is beautiful.
Fact:
The poppy is glad and sluggish.
The following can be determined about the poppy:

ケシが緑色で、かつ熱いなら、そのケシは冷たい。
ケシが青ければ、そのケシは悲しい。
植物が赤ければ、その植物は悪い。
ケシが強ければ、そのケシは古い。
植物が小さければ、その植物は美しい。
事実：
そのケシはうれしく、のろい。
そのケシについて何が導けるか。

正解： Nothing.

新たに導ける属性はない。

[1] Stanford CRFM. SRNScenario — 自然言語の合成課題。
https://github.com/stanford-crfm/helm/blob/v0.4.0/src/helm/benchmark/scenarios/synthetic_reasoning_natural_scenario.py

実例 13 ：自然言語・ hard

問題（全文）

If a plant is green or slow, then the plant is small.
If a poppy is purple, then the poppy is boring.
If a poppy is big, then the poppy is scary.
If a poppy is strong and happy, then the poppy is soft.
If a plant is old, then the plant is red.
Fact:
The poppy is sad and antique.
The following can be determined about the poppy:

正解： The poppy is red.

[1] Stanford CRFM. SRNScenario — 自然言語の合成課題。
https://github.com/stanford-crfm/helm/blob/v0.4.0/src/helm/benchmark/scenarios/synthetic_reasoning_natural_scenario.py

実例 13 ：自然言語・ hard （訳）

問題（全文）の日本語訳

植物が緑色であるか、または遅いなら、その植物は小さい。
ケシが紫色なら、そのケシは退屈なものだ。
ケシが大きければ、そのケシは恐ろしい。
ケシが強く、かつ幸せなら、そのケシは柔らかい。
植物が古ければ、その植物は赤い。
事実：
そのケシは悲しく、古い。
そのケシについて何が導けるか。

正解：ケシは赤い。

[1] Stanford CRFM. SRNScenario — 自然言語の合成課題。
https://github.com/stanford-crfm/helm/blob/v0.4.0/src/helm/benchmark/scenarios/synthetic_reasoning_natural_scenario.py

語彙と生成の補足

新しい属性を導けない問題も含まれる

設定	全生成例	Nothing.	比率 %
easy	11,000	1,899	17.26
hard	11,000	1,910	17.36

既定設定で作った各 11,000 問を集計。規則を適用しても新しい属性が得られなければ、答えは Nothing. となる。

[1] Stanford CRFM. SRNScenario — 自然言語の合成課題。
https://github.com/stanford-crfm/helm/blob/v0.4.0/src/helm/benchmark/scenarios/synthetic_reasoning_natural_scenario.py

hard は、具体語と一般語を対応させる

- 一般属性 25 種に、 big→huge / giant 、 small→tiny / little 、 old→ancient / antique などの語を対応させる。 hard の事実には、こうした具体語を使う。
- 規則と正解には一般語を使う。入力の言い換えを一般属性へ対応させることが、 hard の推論に含まれる。

[1] Stanford CRFM. SRNScenario — 自然言語の合成課題。
https://github.com/stanford-crfm/helm/blob/v0.4.0/src/helm/benchmark/scenarios/synthetic_reasoning_natural_scenario.py

問題を生成し、入力と正解を取り出す

```
from helm.benchmark.scenarios import (
    synthetic_reasoning_scenario as sym,
)
task = sym.SyntheticReasoningScenario(
    "variable_substitution", random_seed=42)
example = task.get_instances("")[6000]
print(example.input.text)
print(example.references[0].output.text)
```

公開実装を入れた Python で実行する。変数置換問題を生成し、先頭の test 例の入力と正解を表示する。

[1] Stanford CRFM. SyntheticReasoningScenario — 記号列の合成課題。
https://github.com/stanford-crfm/helm/blob/v0.4.0/src/helm/benchmark/scenarios/synthetic_reasoning_scenario.py

参考文献

Liang et al. (2023). Holistic Evaluation of Language Models. TMLR.

記号列の操作と、自然言語での 1 段の規則適用を評価する合成課題を提示する。

https://arxiv.org/abs/2211.09110

Wu et al. (2021). LIME: Learning Inductive Bias for Primitives of Mathematical Reasoning. ICML.

演繹・帰納・仮説推論の基本操作を合成データで事前学習し、数学的推論への効果を検証した。

https://proceedings.mlr.press/v139/wu21c.html

参考文献

Clark, Tafjord & Richardson (2020). Transformers as Soft Reasoners over Language. IJCAI.

自然言語で記した規則と事実を Transformer へ与え、推論を言語の処理として扱う枠組みと実験を示した。

https://arxiv.org/abs/2002.05867

Stanford CRFM (n.d.). Synthetic Reasoning 公開生成器。参照 2026-10-19 。

抽象版の 3 課題と自然言語 easy/hard を生成する。短い語彙と規則から、問題文と正解を作る。

https://github.com/stanford-crfm/helm/tree/v0.4.0